

PROJECT REPORT

Project Title: Automatic Water Level Controller

CSE 4202

Embedded System Lab

Section: 8E



Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh
24 September, 2026

Submitted By

1. Rajoanul Islam (11220321009)
2. Subarna Roy (11220320989)
3. Jahanara Sultana Nipa (11220120761)
4. Gazi Enamul Haque Ratul (11220321008)
5. Al Mamun Shaikh (11220321011)

Submitted To

Aishwarja Paul Sourav
Lecturer
Department of Computer Science and Engineering
Northern University of Business and Technology Khulna
Khulna 9208, Bangladesh

Contents

	Page
Cover Page	1
Contents	2
Introduction.....	3
Applications Area and Scope.....	3
Objectives	4
Technology and Tools.....	4
Required Software and Language	4
Hardware Description	5
Connection with ICs / ESP32	6
Working Principle	7
Flowchart and Algorithm.....	8
Data flow from sensors through ICs to I/O devices.....	10
Arduino Code.....	12
Estimated Cost Analysis	15
Responsibilities of Each Members.....	15
Workplan (Gantt Chart)	16
Testing and Results	16
Advantages and Limitations	17
Future Improvement.....	18
Conclusion	18
References.....	19

1. Introduction

Water is one of the most important resources in every home, farm and workplace, yet it is very often wasted. In most buildings an overhead water tank is filled by an electric pump that is switched on and off by hand. If the person forgets to switch the pump off, the tank overflows and both water and electricity are wasted; if the pump is not switched on in time, the tank runs dry. Checking the tank again and again is a boring and unreliable job, especially for busy families.

The proposed system uses an HC-SR04 ultrasonic sensor as the sensing element. It is mounted at the top of the tank and measures the distance to the water surface. The ESP32 acts as the central controller: it reads the sensor, converts the distance to a water level percentage, compares it with programmed lower and upper limits (20% and 90%) and controls the pump through a relay module. The ESP32 also creates its own Wi-Fi hotspot named Water_Controller and hosts a small web dashboard, so the user can see the water level, the pump status and the mode, and can switch between Automatic and Manual mode and turn the pump on or off from a phone or laptop. No internet connection or router is required.

The project demonstrates several fundamental embedded-system concepts: sensor interfacing, pulse-width measurement, threshold-based decision making with hysteresis, digital output control through a relay, power regulation, and a Wi-Fi web server on a microcontroller.

2. Applications Area and Scope

- ✓ **Household Water Tanks:** Automatic filling of overhead and underground tanks in houses and apartment buildings.
- ✓ **Hostels, Offices and Schools:** Places where nobody is responsible for switching the pump on and off.
- ✓ **Farms and Gardens:** Keeping irrigation reservoirs and hydroponic tanks filled between two levels.
- ✓ **Aquariums and Laboratories:** Maintaining the water level of small tanks and laboratory reservoirs.
- ✓ **Continuous Monitoring:** The system measures the level every 1.5 seconds without manual observation.
- ✓ **Local Wi-Fi Control:** The ESP32 works as its own access point, so the system can be used even where there is no router or internet.
- ✓ **Prototype Scope:** The main scope of the prototype is to keep a small tank between 20% and 90% and to provide a simple monitoring and control page.
- ✓ **Educational Purpose:** The system is suitable for learning, demonstration and basic automation applications.
- ✓ **Safety Limitation:** The prototype uses a low-voltage (5 V) pump. For a mains-powered pump, a properly rated relay/contactors, insulation, enclosure and electrical safety measures are required.

- ✓ **Future IoT Integration:** The system can be extended with cloud connectivity, mobile notifications and data logging.

3. Objectives

The main objectives of the project are:

- ✓ To design a low-cost automatic water level controller prototype using an ESP32.
- ✓ To interface an HC-SR04 ultrasonic sensor and measure the water level without touching the water.
- ✓ To automatically start the pump at 20% and stop it at 90% water level.
- ✓ To switch the pump load safely using a relay module and a regulated 5 V supply.
- ✓ To provide a Wi-Fi web dashboard for monitoring the level and for manual control of the pump.
- ✓ To develop a simple and understandable embedded-system algorithm.
- ✓ To test the system under low, medium and high water level conditions.

4. Technology and Tools

SL.	Component	Qty.	Purpose
1	ESP32 DevKit (ESP-WROOM-32)	1	Main controller, Wi-Fi access point and web server
2	HC-SR04 Ultrasonic Sensor	1	Water level (distance) sensing
3	5 V Single-Channel Relay Module	1	Switching the pump
4	Mini Submersible DC Pump	1	Pumping water into the tank
5	LM2596 Buck Converter Module	1	Regulates the battery voltage to 5 V
6	Batteries with holder	1	Power source
7	Jumper wires	As required	Connections
8	Breadboard	1	Prototype assembly
9	Water tank model and tube	1	Water container and water path
10	Smartphone / laptop	1	Access to the web dashboard

5. Required Software and Language

Arduino IDE (with the ESP32 board package installed) is used to write, compile and upload the program to the ESP32. The program uses the WiFi.h library to create the Wi-Fi access point and the WebServer.h library to run the HTTP server on port 80.

The project uses Arduino code written in C++. C++ is a human-readable programming language and is commonly used for microcontroller programming. In the Arduino

environment, a program is called a sketch. The sketch is compiled into machine instructions that can be executed by the microcontroller. The program continuously reads the ultrasonic sensor, calculates the water level, controls the pump according to the selected mode and serves the web dashboard, which is written in HTML and CSS and generated as a text string inside the sketch.

6. Hardware Description

ESP32 DevKit

The ESP32 is a low-cost 32-bit dual-core microcontroller with built-in Wi-Fi and Bluetooth. Its GPIO pins work at 3.3 V logic level. In this design GPIO5 drives the Trig pin and GPIO18 reads the Echo pin of the ultrasonic sensor, and GPIO23 controls the relay. The ESP32 also works as a Wi-Fi access point and web server.

HC-SR04 Ultrasonic Sensor

The HC-SR04 has a transmitter (T) and a receiver (R). When a 10 microsecond HIGH pulse is applied to the Trig pin, the transmitter sends a burst of 40 kHz ultrasonic sound. The sound travels to the water surface and is reflected back. The Echo pin stays HIGH for a time equal to the round-trip travel time of the sound. Since the speed of sound is about 343 m/s (0.0343 cm/μs), the distance is:

$$\text{Distance (cm)} = (\text{Echo time in } \mu\text{s} \times 0.0343) / 2$$

The division by 2 is needed because the measured time covers the trip to the water and back. The sensor works from about 2 cm to 400 cm, which is suitable for a small tank.

Relay Module

A relay is an electrically operated switch. A small current from the ESP32 controls the relay coil and the relay contacts switch the pump circuit, so the ESP32 pin never has to supply the pump current. The relay has three contacts: COM (common), NO (normally open) and NC (normally closed). Here the pump is connected between COM and NO, so it runs only when the relay is energized. The module is active-low: the relay turns ON when its IN pin is pulled LOW.

Mini Submersible Pump

The pump is a small DC motor with an impeller that is placed inside the water source. It works on a low DC voltage (typically 3–6 V) and pushes water through a tube to the tank.

LM2596 Buck Converter and Battery

The LM2596 is a step-down DC-DC converter. It reduces the battery voltage to a stable 5 V, which is set with the blue trimmer potentiometer on the module. This 5 V rail powers the ESP32 (5V pin), the HC-SR04, the relay module and the pump. A switching converter is used instead of a linear regulator because it wastes much less energy as heat.

7. Connection with ICs / ESP32

We have implemented the proposed hardware design using the ESP32 as the main microcontroller. The circuit diagram shows the ESP32, HC-SR04 sensor, relay module, mini pump, LM2596 buck converter and battery holder. The ESP32 receives the sensor input, processes the value and controls the relay.

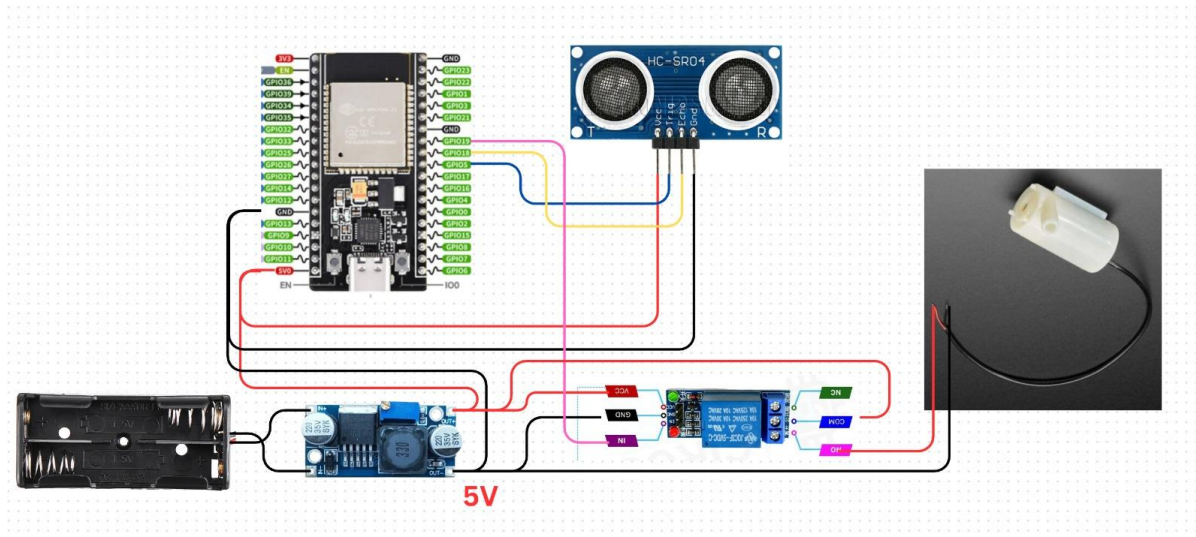


Figure 1: Circuit Diagram of Automatic Water Level Controller

HC-SR04 Connections

HC-SR04 Pin	Connection
Vcc	5 V
Trig	ESP32 GPIO5
Echo	ESP32 GPIO18
Gnd	GND

Relay Module and Pump Connections

Device Pin	Connection
Relay VCC	5 V
Relay GND	GND
Relay IN	ESP32 GPIO23 (active-low)
Relay COM	5 V rail (LM2596 OUT+)
Relay NO	Pump red (+) wire
Pump black (-) wire	GND

Power Connections

Connection	Description
Battery holder + / –	LM2596 IN+ / IN–
LM2596 OUT+ (5 V)	ESP32 5V pin, HC-SR04 Vcc, relay VCC and relay COM
LM2596 OUT– (GND)	ESP32 GND, sensor Gnd, relay GND and pump (–)
All grounds	Common GND

The pump gets its positive supply only through the relay contacts, so when the relay is OFF the pump is completely disconnected from the supply.

8. Working Principle

After power is applied, the ESP32 keeps the pump OFF, starts the Wi-Fi access point and the web server. The program then repeats two jobs continuously. First, it serves the web dashboard to any connected phone or laptop. Second, every 1.5 seconds it triggers the HC-SR04, measures the echo time, and calculates the distance and the water level percentage.

In Automatic mode, if the water level is at or below 20% and the pump is OFF, the ESP32 turns the pump ON. If the level is at or above 90% and the pump is ON, the ESP32 turns the pump OFF. Between 20% and 90% the pump keeps its previous state. In Manual mode the automatic control is skipped and the pump is controlled only by the ON/OFF buttons on the dashboard.

Water Level Calculation

The tank measurements are stored as constants in the program:

Parameter	Value	Meaning
TANK_DEPTH	9.0 cm	Distance from the sensor to the bottom of the empty tank
SENSOR_OFFSET	3.0 cm	Distance from the sensor to the maximum (100%) water level
Effective height	6.0 cm	TANK_DEPTH – SENSOR_OFFSET (usable water height)

$$\begin{aligned} \text{Water height} &= \text{TANK_DEPTH} - \text{Measured distance} \\ \text{Water level (\%)} &= (\text{Water height} / \text{Effective height}) \times 100 \end{aligned}$$

The result is limited to the range 0% to 100%. Some example values are shown below:

Measured distance (cm)	Water level (%)	Pump action (Auto mode)
3.0	100	OFF (tank full)
3.6	90	Turns OFF when reached
6.0	50	No change
7.8	20	Turns ON when reached
9.0	0	ON (tank empty)

Threshold Selection

The lower limit (20%) and the upper limit (90%) are software parameters and can be changed in the code. Using two different limits (hysteresis) instead of a single limit prevents the pump from switching ON and OFF again and again when the water level is close to one value, which protects the relay and the pump.

Sensor Placement and Calibration

TANK_DEPTH and SENSOR_OFFSET must be measured for the actual tank after the sensor is fixed. The sensor should face the water surface straight (not at an angle) and should be kept at least 2 cm above the highest water level. Water ripples, foam, and temperature changes (which change the speed of sound) can slightly change the reading, so the readings should be checked with a scale during testing.

9. Flowchart and Algorithm

The flowchart shows the step-by-step operation of the **Automatic Water Level Controller**. At first, the ESP32 initializes the sensor and pump pins, keeps the pump **OFF**, and starts the **Wi-Fi access point and web server**.

After initialization, the **HC-SR04 ultrasonic sensor** measures the distance between the sensor and the water surface. The ESP32 converts this distance into a **water level percentage** and updates the reading every **1.5 seconds**.

The system then checks the operating mode. In **Automatic Mode**, the ESP32 compares the water level with the lower limit (**20%**) and upper limit (**90%**). If the water level is 20% or below, the pump is turned **ON**. When the water level reaches 90% or above, the pump is turned **OFF**. In **Manual Mode**, automatic control is skipped, and the user can directly turn the pump ON or OFF using the web dashboard.

Finally, the ESP32 repeats the measurement and control process continuously. Thus, the system can continuously monitor the water level and control the pump according to the selected mode.

System Flowchart:

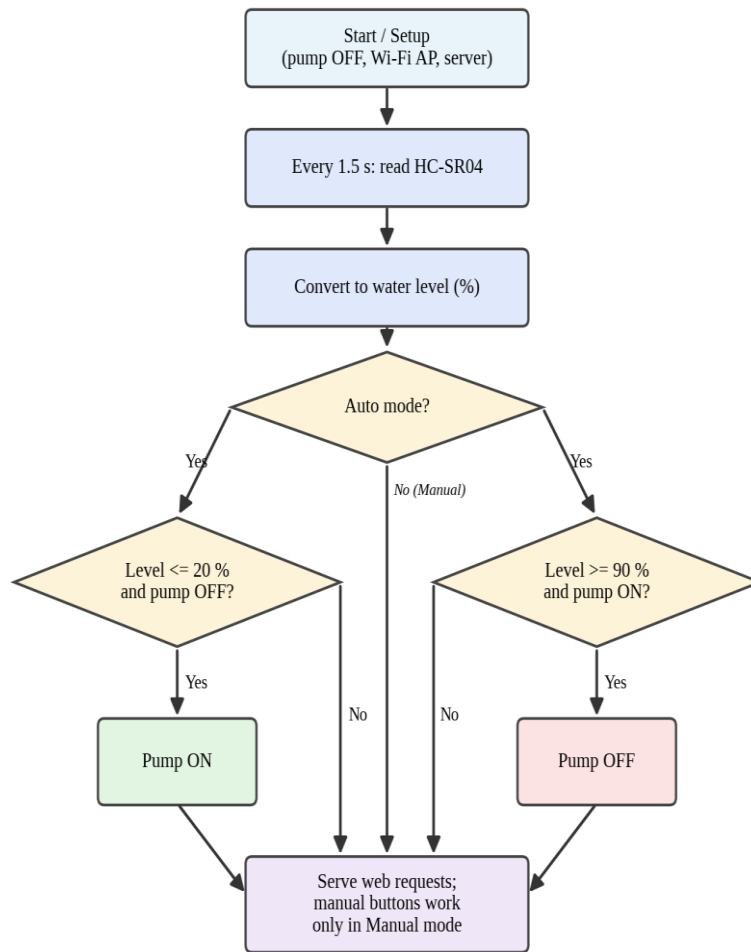


Figure 2: Automatic Water Level Controller Flowchart.

Algorithm

- ❖ Start the system.
- ❖ Initialize the sensor pins, keep the pump OFF, start the Wi-Fi access point and the web server.
- ❖ Handle the web requests from the dashboard continuously.
- ❖ Every 1.5 seconds, trigger the HC-SR04 and measure the echo time.
- ❖ Calculate the distance and convert it to a water level percentage (limited to 0–100%).
- ❖ If Automatic mode: if level $\leq 20\%$ and the pump is OFF, turn the pump ON.
- ❖ If Automatic mode: if level $\geq 90\%$ and the pump is ON, turn the pump OFF.
- ❖ If Manual mode: skip the automatic control; the pump follows the dashboard buttons.
- ❖ Repeat the process continuously.

10. Data flow from sensors through ICs to I/O devices

Data flow from Sensor to ESP32 and ESP32 to the Pump:

The HC-SR04 sends an ultrasonic pulse towards the water surface and receives the echo. The echo time is sent to the ESP32 as a pulse on the Echo pin. The ESP32 converts the time to a distance and then to a water level percentage. According to the level and the selected mode, the ESP32 sends a LOW or high-impedance signal to the relay module, and the relay connects or disconnects the pump from the 5 V rail. The overall data flow can be summarized as:

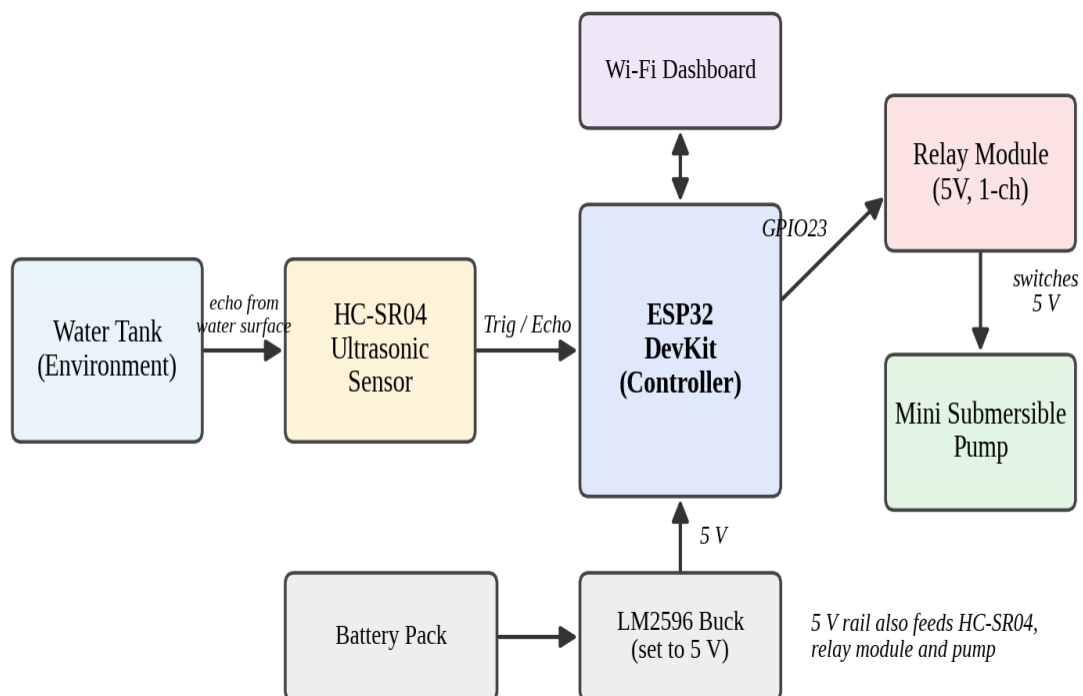
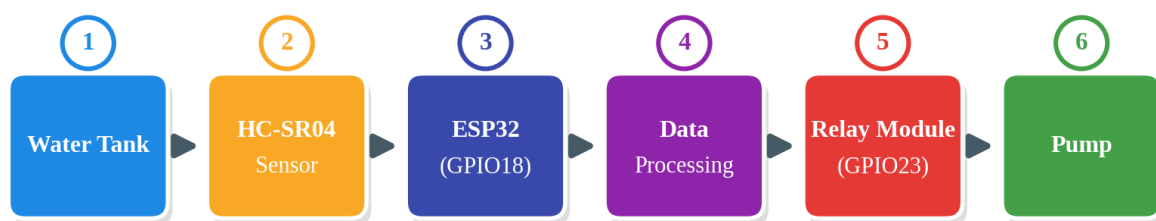


Figure 3: Data flow from Sensor to ESP32 and ESP32 to the Pump.

Data Flow Between ESP32 and the User

The ESP32 works as a Wi-Fi access point with the network name “**Water_Controller.**” The user can connect a mobile phone or laptop to this Wi-Fi network without requiring an external internet connection. After connecting, the user opens a web browser and enters the IP address **192.168.4.1**. The ESP32 then receives the request and sends a web-based dashboard to the user's device.

The dashboard displays important information about the water control system, such as the **current water level, measured distance, operating mode, and pump status**. The user can easily monitor the system from this dashboard. When the user presses a control button, the browser sends a request to the ESP32 through a specific route. For example, the **/toggleMode** route is used to change the operating mode, while **/pumpOn** and **/pumpOff** routes are used to turn the water pump on or off.

After receiving the request, the ESP32 processes the command and performs the required action. It then redirects the browser back to the main dashboard page. The updated pump status, mode, water level, and other information are displayed again. In this way, the ESP32 and the user's device communicate through Wi-Fi, allowing the user to monitor and control the water system easily and in real time.

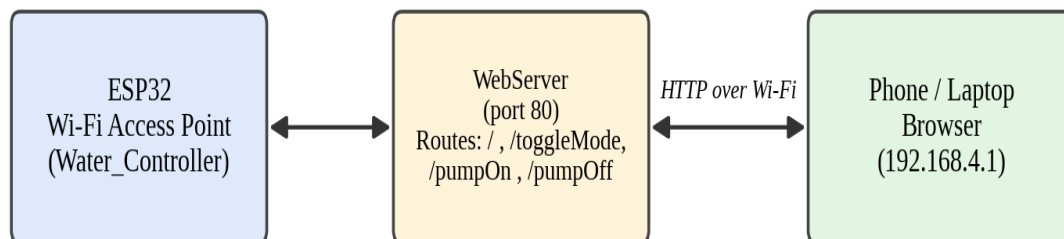


Figure 4: Data flow between the ESP32 and the user's browser.

Thus, the system provides automatic pump control as well as remote monitoring and manual control of the water tank from a phone or laptop.

11. Arduino Code

```
#include <WiFi.h>
#include <WebServer.h>

// Wi-Fi Access Point information
const char *ssid = "Water_Controller";
const char *password = "12345678";

// Pin configuration
#define TRIG_PIN 5
#define ECHO_PIN 18
#define RELAY_PIN 23

// Tank measurement (in centimeters)
const float TANK_DEPTH = 9.0; // Empty tank depth (from sensor to bottom)
const float SENSOR_OFFSET = 3.0; // Distance from sensor to maximum water level

// Control variables
bool autoMode = true;
bool pumpState = false;
float waterLevelPercent = 0.0;
float distanceCM = 0.0;

unsigned long lastSensorRead = 0;
const long interval = 1500;

WebServer server(80);

// Relay control function (Active-Low safety trick)
void setPump(bool state) {
    pumpState = state;
    if (pumpState) {
        pinMode(RELAY_PIN, OUTPUT);
        digitalWrite(RELAY_PIN, LOW); // Relay ON
    } else {
        pinMode(RELAY_PIN, INPUT); // Relay OFF (High-Z)
    }
}

// Ultrasonic reading function
void readUltrasonic() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout
    if (duration > 0) {
        distanceCM = (duration * 0.0343) / 2.0;

        // Calculate water level percentage
        float effectiveHeight = TANK_DEPTH - SENSOR_OFFSET;
        float currentWaterHeight = TANK_DEPTH - distanceCM;
        waterLevelPercent = (currentWaterHeight / effectiveHeight) * 100.0;

        if (waterLevelPercent > 100.0) waterLevelPercent = 100.0;
        if (waterLevelPercent < 0.0) waterLevelPercent = 0.0;
    }
}

// HTML webpage
String getHTML() {
    String html = "<!DOCTYPE html><html><head><meta name='viewport' content='width=device-width, initial-scale=1'>";
    html += "<title>Water Monitor</title>";
    html += "<style>body{font-family:Arial;text-align:center;margin-top:20px;background:#f4f7f6;}";
    html += ".card{background:white;padding:20px;margin:auto;max-width:350px;border-radius:12px;box-shadow:0 4px 10px rgba(0,0,0,0.1);}";
    html += ".btn{padding:12px 24px;font-size:16px;margin:8px;border:none;border-radius:6px;cursor:pointer;color:white;}";
    html += ".btn-on{background:#28a745;}.btn-off{background:#dc3545;}.btn-mode{background:#007bff;}";
}
```

```

    html += ".gauge{font-size:36px;font-weight:bold;color:#17a2b8;margin:15px
0;}</style></head><body>";

    html += "<div class='card'>";
    html += "<h2>Water Monitor</h2>";
    html += "<div class='gauge'>" + String((int)waterLevelPercent) + " %</div>";
    html += "<p>Distance: " + String(distanceCM, 1) + " cm</p>";
    html += "<p>Mode: <b>" + String(autoMode ? "AUTOMATIC" : "MANUAL") + "</b></p>";
    html += "<p>Pump Status: <b>" + String(pumpState ? "RUNNING" : "STOPPED") + "</b></p><hr>";

    html += "<a href='/toggleMode'><button class='btn btn-mode'>Switch Mode</button></a><br>";
    if (!autoMode) {
        if (pumpState) {
            html += "<a href='/pumpOff'><button class='btn btn-off'>Turn Pump OFF</button></a>";
        } else {
            html += "<a href='/pumpOn'><button class='btn btn-on'>Turn Pump ON</button></a>";
        }
    } else {
        html += "<p style='color:gray;'><small>Manual buttons disabled in Auto mode</small></p>";
    }
    html += "</div></body></html>";
    return html;
}

// Web root handler
void handleRoot() {
    server.send(200, "text/html", getHTML());
}

void handleMode() {
    autoMode = !autoMode;
    server.sendHeader("Location", "/");
    server.send(303);
}

void handlePumpOn() {
    if (!autoMode) setPump(true);
    server.sendHeader("Location", "/");
    server.send(303);
}

void handlePumpOff() {
    if (!autoMode) setPump(false);
    server.sendHeader("Location", "/");
    server.send(303);
}

void setup() {
    Serial.begin(115200);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    setPump(false); // Keep the pump OFF at startup

    // Start the Wi-Fi Hotspot (Access Point)
    WiFi.softAP(ssid, password);
    IPAddress IP = WiFi.softAPIP();
    Serial.print("Access Point IP: ");
    Serial.println(IP);

    // Server routing
    server.on("/", handleRoot);
    server.on("/toggleMode", handleMode);
    server.on("/pumpOn", handlePumpOn);
    server.on("/pumpOff", handlePumpOff);

    server.begin();
    Serial.println("Web server started");
}

void loop() {
    server.handleClient();

    // Read the sensor and run automation logic at a fixed interval

```

```

unsigned long currentMillis = millis();
if (currentMillis - lastSensorRead >= interval) {
    lastSensorRead = currentMillis;
    readUltrasonic();

    // Automatic mode logic
    if (autoMode) {
        if (waterLevelPercent <= 20.0 && !pumpState) {
            setPump(true); // Turn the pump ON when the water level falls below 20%
            Serial.println("Auto: Water low! Pump started.");
        }
        else if (waterLevelPercent >= 90.0 && pumpState) {
            setPump(false); // Turn the pump OFF when the water level rises above 90%
            Serial.println("Auto: Water full! Pump stopped.");
        }
    }
}
}
}

```

Code Explanation

- ✓ `setup()` starts the serial monitor, configures the sensor pins, keeps the pump OFF at start-up, starts the Wi-Fi access point, registers the web routes and starts the server.
- ✓ `loop()` handles the web clients continuously and, every 1.5 s (using `millis()` without a blocking delay), reads the sensor and runs the automatic control logic.
- ✓ `readUltrasonic()` generates the trigger pulse, measures the echo time with `pulseIn()` (30 ms timeout), calculates the distance and the water level percentage and limits it to 0–100%.
- ✓ `setPump(bool)` turns the pump ON by driving the relay pin LOW and OFF by setting the pin to INPUT (high impedance), so the relay stays OFF while the ESP32 is booting or restarting.
- ✓ `getHTML()` builds the dashboard page with the latest readings; the manual buttons are shown only in Manual mode.
- ✓ `TANK_DEPTH = 9.0` and `SENSOR_OFFSET = 3.0` are example tank values and must be measured again for the actual tank.
- ✓ Serial Monitor output (“Auto: Water low! Pump started.” / “Auto: Water full! Pump stopped.”) helps to observe the automatic actions during testing.

Web Dashboard Routes

Route	Handler	Action
/	<code>handleRoot()</code>	Sends the dashboard page with level, distance, mode and pump status
/toggleMode	<code>handleMode()</code>	Switches between Automatic and Manual mode
/pumpOn	<code>handlePumpOn()</code>	Turns the pump ON (only if the system is in Manual mode)
/pumpOff	<code>handlePumpOff()</code>	Turns the pump OFF (only if the system is in Manual mode)

12. Estimated Cost Analysis

The following is an estimated cost for physical implementation. Prices may vary depending on seller, component quality, location and market conditions. Transportation and other incidental expenses are not included.

Component	Qty.	Estimated Price (BDT)
ESP32 DevKit	1	600-800
HC-SR04 Ultrasonic Sensor	1	100-150
5 V Single-Channel Relay Module	1	90-130
Mini Submersible DC Pump	1	80-120
LM2596 Buck Converter Module	1	100-150
Batteries with holder	1 set	300-400
Breadboard + wires	1 set	150-250
Water tank model + tube	1	100-150
Total	--	Approx. 1,720–2,300

13. Responsibilities of Each Members

This section can be customized according to the actual contribution of each group member. A suggested distribution is:

- 1.Jahanara Sultana Nipa: Introduction, Applications Area and Scope, Objectives, Workplan (Gantt Chart)
2. Gazi Enamul Haque Ratul: Hardware Description, Circuit diagram, Technology and Tools
3. Rajoanul Islam: ESP32 code, Working Principle, Testing and Results , Connection with ICs / ESP32 (pin descriptions)
- 4.Subarna Roy: Flowchart and Algorithm, Data flow diagrams, Estimated Cost Analysis, report formatting
- 5.Al Mamun Shaikh: Advantages and Limitations, Future Improvement, Conclusion, References

14. Workplan (Gantt Chart)

For this project, the work can be completed in four weeks. In the first week, the project idea is finalized and the components and circuit design are studied. The second week is used for hardware implementation and ESP32 coding. The third week is used for testing, tank calibration and debugging. In the fourth week, final corrections, documentation and presentation preparation are completed.

Task	Week 1	Week 2	Week 3	Week 4
Planning & idea				
Component & circuit				
ESP32 coding				
Hardware implementation				
Testing & calibration				
Debugging				
Report & presentation				

Figure 5: Gantt chart for project implementation

15. Testing and Results

No.	Condition	Expected Output	Result
1	Power ON	Pump stays OFF at start-up; Wi-Fi hotspot Water_Controller appears	Pass / verify
2	Open 192.168.4.1 in browser	Dashboard shows level, distance, mode and pump status	Pass / verify
3	Auto mode, level $\leq 20\%$	Pump turns ON automatically	Pass / verify
4	Auto mode, level between 20% and 90%	Pump keeps its previous state (no repeated switching)	Pass / verify
5	Auto mode, level $\geq 90\%$	Pump turns OFF automatically	Pass / verify
6	Press “Switch Mode” (Manual)	Manual ON/OFF buttons appear; no automatic switching	Pass / verify
7	Manual mode, press pump buttons	Pump follows the button (ON / OFF)	Pass / verify
8	Serial Monitor	Access point IP and Auto messages are displayed	Pass / verify

Expected Result

When the system is powered on, the pump remains OFF and the dashboard becomes available. In Automatic mode, when the water level falls to 20% or below, the pump starts and fills the tank; when the level reaches 90% or above, the pump stops. In Manual mode the pump is controlled only from the dashboard. The readings shown on the dashboard should be compared with a scale to check the tank constants.

16. Advantages and Limitations

Advantages

- ❖ Low-cost and easy to build.
- ❖ Fully automatic filling between two levels; prevents overflow and dry running (in Auto mode).
- ❖ Non-contact water level sensing; the sensor does not touch the water.
- ❖ Works without internet or router because the ESP32 creates its own Wi-Fi hotspot.
- ❖ Web dashboard shows the level and allows manual control from a phone or laptop.
- ❖ Relay isolates the pump from the ESP32; pump stays OFF during boot and restart.
- ❖ Limits can be modified through software.
- ❖ Suitable for embedded-systems learning and prototype demonstrations.

Limitations

- ✓ The Echo pin of the HC-SR04 gives a 5 V signal, while the ESP32 GPIO pins are 3.3 V devices. A voltage divider (for example 1 k Ω and 2 k Ω) on the Echo line will protect the ESP32.
- ✓ The LM2596 needs an input voltage higher than its 5 V output (at least about 7 V), so the battery pack must be chosen accordingly, for example two 18650 Li-ion cells in series.
- ✓ Ultrasonic readings are affected by temperature, foam, ripples, and the shape of the tank; TANK_DEPTH and SENSOR_OFFSET must be measured for each tank.
- ✓ The dashboard does not refresh by itself; the page has to be reloaded to see a new reading.
- ✓ If the sensor gives no echo (timeout), the last level value is kept and there is no sensor-fault protection or dry-run protection.
- ✓ In Manual mode there is no automatic stop, so the tank can overflow if the user forgets to turn the pump off.
- ✓ The default Wi-Fi password should be changed to a stronger one. The prototype uses a low-voltage pump and is not a certified product for mains-powered pumps.

17. Future Improvement

Several improvements can make the system more useful and robust:

- ❖ Add automatic refresh of the dashboard and a real-time graph of the water level.
- ❖ Connect the ESP32 to the home Wi-Fi/cloud and send mobile notifications for low or high water level.
- ❖ Store water level readings in a database or cloud dashboard.
- ❖ Add a sensor-fault check, dry-run protection and a maximum pump run time.
- ❖ Use a waterproof ultrasonic sensor (for example JSN-SR04T) for humid environments.
- ❖ Add a backup float switch as a hardware safety layer against overflow.
- ❖ Add a voltage divider on the Echo line and a stronger Wi-Fi password.
- ❖ Use a properly rated relay/contactors and enclosure for mains-powered pumps.
- ❖ Add battery backup and power-failure indication.

18. Conclusion

The Automatic Water Level Controller successfully demonstrates how a microcontroller can be combined with an ultrasonic sensor, a relay module and a pump to create a basic real-time water management prototype. The HC-SR04 provides the input signal, the ESP32 performs processing and decision making, the relay switches the pump, and the Wi-Fi dashboard communicates the system status and allows manual control.

The project is particularly useful for understanding sensor interfacing, pulse-width measurement, threshold-based control with hysteresis, relay switching, power regulation and web-server programming on a microcontroller. Although the prototype is simple, the same basic architecture can be extended with cloud connectivity, data logging, notifications and fault protection to create a more capable smart water management system.

The most important practical considerations are the calibration of the tank constants, the correct 5 V supply and the 3.3 V protection of the Echo pin. The values used in the demonstration should be treated as a starting point for the actual tank.

19. References

1. Espressif Systems, “Arduino-ESP32 Documentation (WiFi and WebServer libraries)”.
<https://docs.espressif.com/projects/arduino-esp32/en/latest/>
2. Espressif Systems, “ESP32 Series Datasheet”.
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
3. Cytron / SparkFun, “HC-SR04 Ultrasonic Sensor Datasheet”.
<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
4. Texas Instruments, “LM2596 SIMPLE SWITCHER Power Converter Datasheet”.
<https://www.ti.com/lit/ds/symlink/lm2596.pdf>
5. Random Nerd Tutorials, “ESP32 with HC-SR04 Ultrasonic Sensor with Arduino IDE”.
<https://randomnerdtutorials.com/esp32-hc-sr04-ultrasonic-arduino/>
6. Random Nerd Tutorials, “How to Set an ESP32 Access Point (AP) for Web Server”.
<https://randomnerdtutorials.com/esp32-access-point-ap-web-server/>
7. Random Nerd Tutorials, “ESP32 Relay Module – Control AC Appliances (Web Server)”.
<https://randomnerdtutorials.com/esp32-relay-module-ac-web-server/>
8. Arduino Documentation, “pulseIn() – Advanced I/O Reference”.
<https://www.arduino.cc/reference/en/language/functions/advanced-io/pulsein/>
9. Arduino Documentation, “millis() – Time Reference”.
<https://www.arduino.cc/reference/en/language/functions/time/millis/>
10. Espressif Systems, “arduino-esp32: Arduino core for the ESP32” (GitHub repository).
<https://github.com/espressif/arduino-esp32>